

ARbnb: Markerless AR Indoor Navigation and Spatial Annotation for Short-Term Rental Guests

Team Musashi

Kartik Singhal
Trinity College Dublin
Dublin, Ireland
singhaka@tcd.ie

Jayananden
Meenakchisundaram
Trinity College Dublin
Dublin, Ireland
meenakcj@tcd.ie

Diksha Chottani
Trinity College Dublin
Dublin, Ireland
chottand@tcd.ie

Abstract

We present **ARbnb**, a mobile augmented reality system that guides Airbnb guests through an unfamiliar rental properties using floating waypoint arrows and annotation panels triggered via proximity and anchored to real-world objects. The system comprises three integrated components: (i) a Unity 6 guest application built on AR Foundation and ARCore, (ii) a React and Three.js host dashboard for spatial annotation authoring and (iii) a Node.js with PostgreSQL backend. The spatial annotations persist across sessions using the Google ARCore Cloud Anchors which let a device host an anchor to the cloud and allow subsequent devices to resolve an identical reference frame from the same unique identifier [Google LLC(2025)]. The architecture purposely avoids the use of physical markers and LiDAR dependency, instead relying entirely on markerless SLAM-based tracking [Raccoons NV(2019)]. In this project, we demonstrate a functional end-to-end prototype which seamlessly handovers from host authoring on a desktop browser to in-situ guest navigation on a Samsung Galaxy S25 Ultra running Android 15.

Keywords

Augmented Reality, Indoor Navigation, Spatial Annotation, ARCore, Cloud Anchors, Unity, Short-Term Rental

1 Introduction

The short-term rental properties presents a distinct navigation challenge which the larger hospitality venues have only partially addressed. Unlike hotels, Airbnb properties varies enormously in layout, appliance configuration and house specific procedures. The conventional handover mechanisms like PDF guides, printed notes or lengthy text messages are spatially detached and a guest must mentally map written instructions onto a physical environment which they have never seen. Research in hospitality AR confirms that digital overlays anchored to the physical environment reduce wayfinding friction and improve guest orientation significantly [ProStay(2025), Mews(2025)].

The prior work on AR indoor navigation has explored marker-based, beacon-based and SLAM-based approaches. The marker-based systems require prepared fiducials in the environment; beacon-based systems require Bluetooth or WiFi infrastructure installation. Both impose upfront physical cost that is impractical in a transient rental context [Raccoons NV(2019),

Lopez Mendez(2018)]. Whereas, Markerless SLAM, as implemented in Google ARCore offers an infrastructure free alternative that works in any textured indoor environment and tracks position within 1 to 2 cm error under typical conditions [Lopez Mendez(2018)].

Our application, ARbnb makes three contributions:

- (1) A two-phase Cloud Anchor workflow that provides immediate annotation visibility while progressively upgrading spatial precision across guest sessions.
- (2) A dual-mode host authoring interface combining 2D floor-plan and 3D model views, allowing precise annotation placement without AR hardware.
- (3) A coordinate-handedness bridge between a Three.js right-handed authoring environment and Unity’s left-handed AR runtime.

2 Related Work and Background

2.1 Markerless AR Indoor Navigation

ARCore’s SLAM-based motion tracking uses camera feature points augmented by IMU data to maintain a consistent spatial map which makes it suitable for indoor environments without infrastructure [Raccoons NV(2025)]. The evaluation studies confirm improved usability and orientation compared to paper maps in indoor settings. The hybrid approaches combining SLAM with sparse AR markers have been proposed for feature poor regions though purely markerless setups are preferable in rental contexts where hosts cannot alter the physical environment or space.

2.2 Persistent and Shared AR Anchors

Google ARCore Cloud Anchors enable shared and persistent AR experiences by uploading a local anchor’s feature point data to Google’s cloud endpoint and returns a *cloud anchor ID* that any device in the same physical location can use to resolve an identical anchor pose [Google LLC(2025), Google LLC(2026)]. The persistent Cloud Anchors supports a lifetime of up to 365 days which makes them well suited to the rental scenario where a spatial annotation authored once must be discoverable by every future guest without re-scanning.

2.3 AR in Hospitality

The hospitality industry has applied AR to wayfinding in hotels and resorts where large or complex layouts benefit from directional overlays and contextual facility information [ProStay(2025),

Mews(2025), Flam AI(2025)]. These deployments primarily target hotel-scale properties with managed infrastructure. Our application, ARbnb extends this concept to the unmanaged, heterogeneous environment of short-term rentals, where infrastructure deployment is infeasible and authoring must be done remotely by non-technical hosts.

3 System Design and Implementation

ARbnb comprises three tightly integrated components as shown in Figure 1. The host dashboard handles spatial authoring whereas the Node.js backend mediates persistence and session management and the Unity guest app delivers the in-situ AR experience resolving spatial anchors directly from Google Cloud Anchors at runtime.

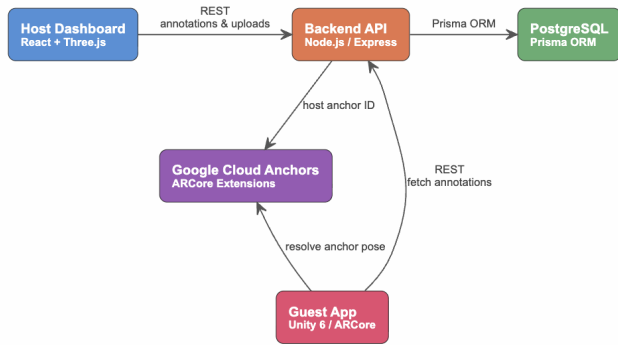


Figure 1: ARbnb system architecture: data flow between the host dashboard, backend API, PostgreSQL database, Google Cloud Anchors, and the Unity guest application.

3.1 AR Guest Application

The guest app targets Android 15 on the Samsung Galaxy S25 Ultra and is built with Unity 6, AR Foundation 6.4.1, ARCore XR Plugin 6.4.1, URP 17.4.0 and XR Interaction Toolkit 3.4.0. All the ARbnb runtime scripts ARbnbSession, AnnotationManager, Waypoint-Navigator, DiscoveryHUD and CloudAnchorService—reside on a single ARbnb GameObject within the main scene.

Annotation rendering: AnnotationPanel builds the floating annotation cards procedurally where a quad mesh as the background card with two 3D TextMeshPro components for title and body text. This avoids Unity’s World Space Canvas entirely, which fails to render reliably in Unity 6 URP AR contexts. A pre-authored URP Unlit material is assigned via [SerializeField] which ensures the shader is included in the build. Relying on Shader.Find() at runtime or Unity’s default material causes objects to render pink or purple in URP builds.

Proximity culling: At most three annotation panels are visible simultaneously which are sorted by distance from the camera to prevent visual clutter in annotation dense areas.

Coordinate handedness: All the API coordinates stored in Three.js (right-handed, Y-up) are converted to Unity (left-handed, Y-up) by negating the Z component,

Listing 1: Handedness conversion in AnnotationManager.cs

```
var unityPos = new Vector3(worldX, worldY, -worldZ);
```

3.2 Cloud Anchor Lifecycle

The two phase approach ensures the annotation experience is never blocked waiting for cloud synchronisation.

Phase 1 – First guest (hosting): Panels appear immediately at raw worldX/Y/Z coordinates. In parallel,

CloudAnchorService.HostAnchorAsync(Vector3) calls the AR-Core Extensions hosting the backend API. On success, the resulting cloudAnchorId is persisted to the database via method POST /api/anchors.

Phase 2 – Subsequent guests (resolving): The panels again appear at raw positions first. AnnotationManager then calls ResolveAnchorAsync(cloudAnchorId) for each annotation. Once each ARAnchor resolves, AnnotationPanel.SnapToAnchor(Transform) re-parents the panel to the resolved transform with correcting positional drift from the raw world space estimate.

Fallback: If incase, the CloudAnchorService is not wired and the manager skips cloud sync entirely where raw positions from the local ARAnchorManager serve as fallback and preserves the functionality without internet access.

3.3 Host Dashboard

The dashboard is a React 18 with Vite 8 single-page application with API requests proxied to the backend. For authentication it uses localStorage-stored user IDs sent as Bearer tokens and no cloud identity provider is required.

FloorPlanEditor: An orthographic Three.js canvas allows the hosts to click a loaded floor plan image to place annotation pins where the positions are stored as normalised floorX/Y coordinates (0 to 1) and independent of image resolution.

ModelViewer: A perspective canvas which supports orbit mode and pin placement mode for 3D GLB/GLTF models via GLTFLoader and DRACOLoader. This viewer captures worldX/Y/Z in metres, which map directly to AR world space after the Z-negation transform described above. Here, the models should be exported from Blender with Y-up orientation and metre scale per the glTF 2.0 specification.

3.4 Backend API

The backend uses Express 4, Prisma 5, PostgreSQL 16, and multer 2. It exposes endpoints for property CRUD operations, floor plan and model file uploads (100 MB limit), annotation management, Cloud Anchor ID storage and guest session lifecycle. The uploaded files are stored in backend/uploads/ and served via Express static middleware. The database schema comprises four Prisma models: Property, Annotation, CloudAnchor and Session. Annotation stores both 2D floor-plan coordinates (floorX, floorY) and 3D world-space coordinates (worldX, worldY, worldZ) supporting both authoring modes.

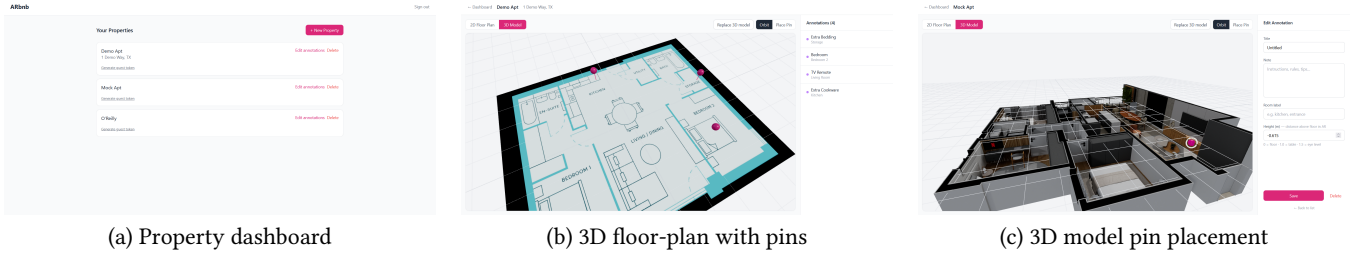


Figure 2: Host authoring workflow: (a) property management dashboard, (b) annotation pins placed on an extruded 3D floor plan, and (c) pin placement on a GLB model with the annotation edit panel.

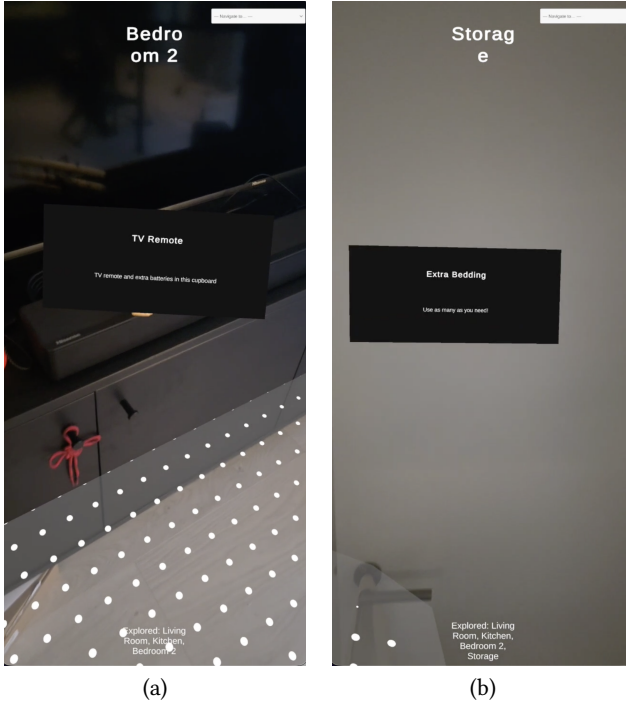


Figure 3: ARbnb guest app running on Samsung Galaxy S25 Ultra (Android 15): (a) proximity-triggered annotation panel anchored to a TV unit, with the discovery HUD showing explored rooms; (b) spatial annotation panel anchored to a storage room wall.

4 Experimental Results

We evaluate the prototype qualitatively across the complete end-to-end pipeline on the Samsung Galaxy S25 Ultra (Android 15, ARCore). Formal quantitative evaluation is planned as future work.

End-to-end pipeline: A host created a property in the dashboard, uploaded a Blender-exported GLB model, placed three spatial annotations in the 3D viewer, and generated a guest session token. The guest app retrieved annotations over the ADB reverse tunnel, rendered all three floating panels in AR, and correctly culled to the nearest three when all were simultaneously in range.

Two-phase anchor behavior: On first launch, annotation panels appeared at raw world coordinates within one second of fetch

completion. Cloud Anchor hosting completed within 4-7 seconds; the cloudAnchorId was then stored to the database. On a second launch simulating a returning guest, panels appeared immediately and subsequently snapped to the resolved anchor pose, demonstrating the progressive precision improvement described in Section 3.2.

Rendering correctness: World-space annotation cards rendered correctly with pre-authored URP Unlit materials. A negative test confirmed that omitting the [SerializeField] material assignment caused the expected pink rendering, validating the design rationale.

Coordinate mapping: Annotations placed in the Three.js dashboard at known 3D coordinates appeared at the correct corresponding positions in the AR scene after Z-negation, confirming the coordinate conversion is both necessary and sufficient.

Table 1: ARbnb prototype system properties

Property	Value
AR framework	AR Foundation 6.4.1 / ARCore 6.4.1
Render pipeline	URP 17.4.0 (Performant profile)
Tracking	Markerless ARCore SLAM
Persistence	Google Cloud Anchors (up to 365 days)
Max visible panels	3 (distance-sorted)
API endpoints	16+ REST endpoints
DB entities	4 (Prisma / PostgreSQL 16)
Upload limit	100 MB per floor plan / model
Test device	Samsung Galaxy S25 Ultra, Android 15

5 Discussion and Conclusion

Design strengths: The separation between host authoring and guest consumption is the most consequential architectural decision. Hosts work in familiar 2D/3D browser tools without requiring AR hardware or physical access during setup. Guests receive spatially grounded instructions requiring zero prior knowledge of the property layout. The dual-mode authoring interface floor plan for room-level guidance, 3D model for object-level annotations, offers progressive precision that suits both quick deployments and detailed property setups.

The two-phase anchor strategy is well-suited to deployment. Requiring a fully resolved cloud anchor before displaying any content would block the experience when the cloud service is slow or

unavailable. Showing annotations at raw coordinates first and then snapping them to the resolved anchor means the worst-case experience remains useful, while subsequent visits benefit from calibrated spatial alignment [Google LLC(2025), Google LLC(2026)].

Limitations: Cloud Anchor hosting quality is sensitive to environmental texture: feature-poor rooms may produce unreliable anchors [Google LLC(2026)]. The prototype has not undergone formal user evaluation; timing data, spatial error measurements, and comparative studies against paper-based instructions are outstanding. The current authentication model is intentionally minimal and must be replaced for any real deployment.

Future work: Planned enhancements include:

- formal guest usability studies measuring task completion time and navigation error
- multimedia annotations supporting images and short video clips;
- cloud file storage and production authentication;
- audio narration for accessibility;
- outdoor zone guidance via the ARCore Geospatial API.

Conclusion. ARbnb demonstrates that a full-stack AR navigation and spatial annotation system for short-term rentals can be

built from commodity components. The system delivers a coherent host-to-guest pipeline, validates the two-phase Cloud Anchor approach for progressive spatial persistence, and identifies the key engineering challenges coordinate handedness, URP material compatibility, and anchor quality that must be addressed for robust real-world deployment.

References

- [Flam AI(2025)] Flam AI. 2025. *Augmented Reality in Hotels: Enhancing the Hospitality Industry*. <https://flamapp.ai/blog/augmented-reality-in-hotels>
- [Google LLC(2025)] Google LLC. 2025. *Cloud Anchors developer guide for Unity (AR Foundation)*. <https://developers.google.com/ar/develop/cloud-anchors>
- [Google LLC(2026)] Google LLC. 2026. *ARCore Cloud Anchors with persistent Cloud Anchors*. <https://codelabs.developers.google.com/codelabs/arcore-cloud-anchors#0>
- [Lopez Mendez(2018)] Roberto Lopez Mendez. 2018. *Indoor Real-Time Navigation with SLAM on Your Mobile*. <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/indoor-real-time-navigation-with-slam-on-your-mobile>
- [Mews(2025)] Mews. 2025. *How augmented reality (AR) is used in tourism and the hotel sector*. <https://www.mews.com/en/blog/augmented-reality-ar-tourism>
- [ProStay(2025)] ProStay. 2025. *AR in the Hospitality Industry: Guest Experiences Through Augmented Reality*. <https://www.prostay.com/blog/ar-in-hospitality-industry/>
- [Raccoons NV(2019)] Raccoons NV. 2019. *Creating an ARCore Powered Indoor Navigation Application in Unity*. <https://www.raccoons.be/what-we-think/articles/creating-an-arcore-powered-indoor-navigation-application-in-unity>